Implementação e Validação Funcional de um Processador MIPS Monociclo com Suporte a Instruções de Salto e Execução do Algoritmo Sort

José Dercy Vieira da Silva Filho
Faculdade de Engenharia Elétrica e da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
jose.filho@ufam.edu.br

Vinicius Menezes
Faculdade de Engenharia Elétrica e da Computação (FEEC)
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
vinicius.menezes@ufam.edu.br

***Resumo*—Este artigo apresenta o projeto, a modelagem em Verilog HDL parametrizado conforme a norma IEEE Std 1364-2005, a síntese lógica em FPGA Intel MAX 10 e a validação funcional de um processador MIPS Monociclo de 32 bits com arquitetura Harvard. O caminho de dados foi projetado para executar fluxos sequenciais, desvios condicionais `beq` e `bne`, e saltos incondicionais e de sub-rotina `j`, `jal` e `jr` em um único ciclo de relógio. A homologação da microarquitetura ocorreu sob a carga real do algoritmo de ordenação de vetores `sort.bin`, montado pelo simulador MARS. Para viabilizar a execução do binário sem qualquer alteração no código de máquina, o hardware recebeu uma máscara combinacional de endereçamento na memória ROM através do barramento `addr[9:2]` e um inicializador de registradores no momento do reset. A metodologia de validação compreendeu uma suíte automatizada de 7 bancadas de teste em Icarus Verilog com 100% de taxa de aprovação e ordenação na memória RAM. A síntese lógica no Intel Quartus Prime alcançou zero erros de elaboração para a placa Terasic DE10-Lite.**

***Index Terms*—MIPS Monociclo, Verilog HDL, Instruções de Salto, Algoritmo Sort, Bateria de Testbenches, FPGA Intel MAX 10.**

I. Introdução

A implementação em silício e em linguagem de descrição de hardware de arquiteturas RISC constitui base essencial da engenharia de computação [1]. O conjunto de instruções MIPS32 destaca-se pelo formato ortogonal de 32 bits, decodificação regular e simplicidade dos blocos funcionais [2], [3].

Na microarquitetura monociclo cada instrução é buscada na memória de instruções, decodificada pela unidade de controle, executada na ULA, acessa a memória de dados e grava o resultado no banco de registradores em um único ciclo de relógio com CPI igual a 1. O período do relógio acomoda o caminho crítico do sistema, definido pela latência acumulada dos multiplexadores, da ULA e dos acessos às memórias síncronas e assíncronas [1].

No âmbito da Segunda Avaliação Parcial da disciplina, este trabalho atende aos seguintes objetivos centrais:

1) Caminho de dados com saltos: suporte completo a desvios condicionais `beq` e `bne`, saltos incondicionais diretos `j`, chamadas com link `jal` e retornos indiretos por registrador `jr`;
2) Parametrização RTL do circuito sob a norma IEEE Std 1364-2005 com barramento `DATA_WIDTH = 32` [4];
3) Validação funcional individual de cada módulo e da integração top-level por bancadas de teste automatizadas com assertivas;
4) Homologação do processador monociclo executando o código de máquina do procedimento de ordenação `sort.bin` [1], derivado do exemplo canônico em C e assembly do livro-texto da disciplina, demonstrando leitura, manipulação e reescrita de dados na memória RAM;
5) Síntese e compilação para o dispositivo FPGA Intel MAX 10 da placa Terasic DE10-Lite [5] via software Intel Quartus Prime [6].

O restante deste artigo organiza-se da seguinte forma: a Seção II descreve o ambiente de desenvolvimento e automação; a Seção III detalha a especificação arquitetural e as soluções de hardware para o algoritmo Sort; a Seção IV apresenta os resultados dos testbenches e a execução na RAM; e a Seção V apresenta as considerações finais e os trabalhos futuros.

II. Ambiente de Desenvolvimento e Automação

O fluxo de engenharia digital adotado combina ferramentas de síntese para lógica programável e utilitários abertos de simulação em ambiente Linux, estruturados em uma esteira de automação por linha de comando.

A. Plataforma de Síntese e Dispositivo Alvo

A síntese lógica e o mapeamento tecnológico foram configurados no software Intel Quartus Prime Standard/Lite Edition [6]. O dispositivo físico de destino é o FPGA Intel MAX 10 modelo `10M50DAF484C7G`, presente na placa de desenvolvimento Terasic DE10-Lite [5]. Este dispositivo possui 50.000 elementos lógicos, blocos de memória embarcada M9K de 432 Kibit e blocos multiplicadores dedicados de 18×18.

O processo de compilação física divide-se em três etapas acionadas via terminal:

1) `quartus_map`: análise sintática e síntese lógica em portas genéricas independentes de tecnologia;

2) `quartus_fit`: posicionamento e roteamento nos recursos de silício e canais de interconexão física do MAX 10;
3) `quartus_asm`: geração do arquivo binário de configuração final em formato `.sof`.

A gravação física na placa é realizada via interface JTAG com o gravador USB-Blaster integrado, utilizando o utilitário `quartus_pgm` após detecção com `jtagconfig`.

B. Ferramentas de Simulação e Verificação RTL

Para acelerar o ciclo de depuração temporal, empregou-se o compilador de simulação Icarus Verilog através do comando `iverilog` e seu motor de execução `vvp` [7]. As formas de onda dos barramentos e sinais de controle foram exportadas para arquivos em formato VCD e inspecionadas graficamente no visualizador GTKWave.

C. Automação via Makefile e Scripts de Regressão

Todo o fluxo de compilação, simulação e síntese foi encapsulado em um `Makefile` centralizado e scripts Bash estruturados no diretório `scripts/`:

- `make test`: executa o script `run_all_tests.sh`, disparando a regressão sobre as 7 bancadas com verificação de status;
- `make top`: simula o processador executando o código de máquina `sort.bin`;
- `make quartus`: executa a análise estrutural rápida de síntese e elaboração;
- `make quartus-build`: conduz a compilação completa até a geração do bitstream `.sof`.

Essa automação garante reprodutibilidade estrita em ambientes de integração contínua, eliminando dependências manuais de telas gráficas.

III. Especificação Arquitetural e Adaptações de Hardware

A microarquitetura do processador MIPS monociclo adota o modelo Harvard com memórias e barramentos estritamente independentes para instruções (ROM) e dados (RAM) [1], como ilustrado no netlist de topo gerado pelo sintetizador na Fig. 5. Em uma topologia monociclo com tempo de instrução unitário (CPI $= 1$), a separação de memórias constitui uma restrição física absoluta: instruções de leitura e escrita em memória (`lw` e `sw`) realizam a busca da instrução e o acesso ao dado no mesmo ciclo de relógio. Uma arquitetura Von Neumann com memória unificada sofreria um conflito estrutural insolúvel (*structural hazard*), inviabilizando a execução monociclo sem ciclos de espera.

A. Caminho de Dados Parametrizado (Datapath)

O caminho de dados em `datapath.v` adota parametrização integral sob a norma IEEE Std 1364-2005 [4], com parâmetros de largura `DATA_WIDTH` e `ADDR_WIDTH` fixados em 32 bits. Essa definição propaga-se estruturalmente aos submódulos (`pc.v`, `adder.v`, `alu.v`, `reg_file.v` e `mux2to1.v`), cujos barramentos derivam da expressão `[DATA_WIDTH-1:0]` (Fig. 1).

Cabe ressaltar que os campos estruturais de decodificação da instrução, como opcode (`[31:26]`), índices de registradores (`[25:21]`, `[20:16]`, `[15:11]`), `shamt` (`[10:6]`) e `funct` (`[5:0]`), representam constantes normativas invariantes da arquitetura MIPS32 [1]. Alterá-los comprometeria a conformidade com compiladores e montadores padrão.

```
module datapath #(
    parameter DATA_WIDTH = 32,
    parameter ADDR_WIDTH = 32
)(
    input  wire                  clk, reset,
    input  wire [DATA_WIDTH-1:0] instr,
    input  wire [DATA_WIDTH-1:0] mem_read_data,
    output wire [ADDR_WIDTH-1:0] pc_out,
    output wire [ADDR_WIDTH-1:0] alu_result,
    output wire [DATA_WIDTH-1:0] mem_write_data,
    output wire [5:0]            opcode, funct,
    output wire                  zero
);
```

Figura 1. Declaração parametrizada da entidade topológica do Datapath (`datapath.v`).

Os blocos fundamentais do caminho de dados articulam-se da seguinte forma:

- Banco de registradores (`reg_file.v`): matriz parametrizada de 32 registradores de 32 bits com duas portas de leitura combinacionais e escrita síncrona na borda positiva do clock. O registrador \$0 é fixado permanentemente em zero;
- Unidade lógica e aritmética (`alu.v`): módulo combinacional que executa operações aritméticas e lógicas. A subtração reaproveita o somador computando $A + \overline{B} + 1$, a comparação `slt` opera via avaliação com sinal (`$signed`), e o deslocamento `sll` opera pelo campo `shamt` ($B \ll$ shamt), conforme ilustrado na Fig. 2;
- Sinalizador de zero: computado por redução unária rápida zero $= \sim |$ alu_result, determinando a condição de branch;
- Extensor de sinal (`sign_extend.v`): converte imediatos de 16 bits para 32 bits replicando o bit mais significativo.

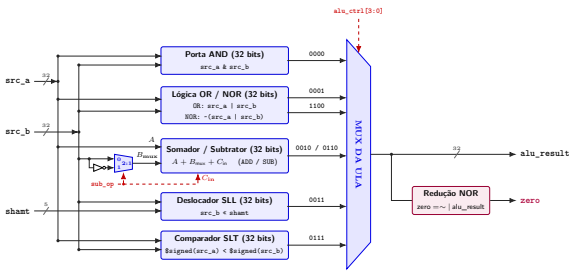


Figura 2. Esquemático estrutural da Unidade Lógica e Aritmética (ULA) de 32 bits com multiplexação de operações e sinalizador de zero.

A restrição temporal da microarquitetura monociclo impõe que o período mínimo de relógio (T_{clk}) acomode a propagação cumulativa da instrução de maior latência do conjunto (`lw`),

definida analiticamente por:

$$T_{\text{clk}} \geq t_{\text{pc}} + t_{\text{rom}} + t_{\text{rf_read}} + t_{\text{alu}} + t_{\text{ram}} + t_{\text{mux}} + t_{\text{rf_setup}} \quad (1)$$

onde os termos denotam as latências do contador de programa, ROM, leitura do banco de registradores, ULA, RAM, multiplexadores e setup de escrita. Essa relação constitui uma restrição física absoluta que limita a frequência máxima de operação (F_{max}) do monociclo, fundamentando a necessidade da divisão em múltiplos estágios no processador em pipeline.

B. Lógica de Seleção de Próximo PC e Saltos

No módulo `datapath.v`, a seleção do endereço da próxima instrução é materializada pela instanciação estrutural dos multiplexadores `mux_branch`, `mux_jump` e `mux_jr` em cascata de prioridade. Esse circuito conecta as saídas intermediárias `pc_next_branch` e `pc_next_jump` até convergir no barramento final `next_pc`, conforme esquematizado na Fig. 3.

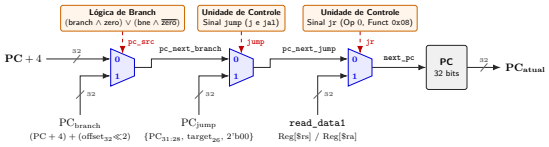


Figura 3. Esquemático da lógica de seleção do próximo PC em cascata no caminho de dados (`beq`, `bne`, `j`, `jal` e `jr`).

A lógica de decisão obedece à seguinte estrutura hierárquica:

1) Desvio condicional com instruções `beq` e `bne`:
$$\text{pc_src} = (\text{branch} \wedge \text{zero}) \vee (\text{bne} \wedge \overline{\text{zero}}) \quad (2)$$
Seleciona entre $\text{PC} + 4$ e o endereço de branch $(\text{PC} + 4) + (\text{offset}_{32} \ll 2)$;
2) Salto incondicional e chamada com link com instruções `j` e `jal`: o multiplexador seleciona entre o estágio de branch e o endereço de salto formado por:
$$\text{PC}_{\text{jump}} = \{\text{PC} + 4[31:28],\ \text{target}[25:0],\ 2\text{'b00}\} \quad (3)$$
Na instrução `jal`, o endereço de retorno $\text{PC} + 4$ é gravado no registrador \$ra via multiplexador com controle $\text{mem_to_reg} = 2\text{'b10}$ e $\text{reg_dst} = 2\text{'b10}$;
3) Retorno de função com instrução `jr`: o multiplexador final seleciona entre o resultado dos estágios anteriores e o dado do registrador `read_data1` via sinal `jr`, garantindo prioridade no fechamento do ciclo.

A unidade de controle possui uma trava de segurança combinacional para o sinal de escrita:

$$\text{reg_write} = \text{reg_write_wire} \wedge \overline{\text{jr}} \quad (4)$$

Essa porta lógica impede que a instrução de retorno sobrescreva o registrador \$ra com dados residuais da ULA.

C. Engenharia de Integração para o Procedimento de Ordenação

A execução do código de máquina `sort.bin`, compilado a partir do assembly `sort.s` com base no procedimento canônico de ordenação de Patterson e Hennessy [1], exigiu soluções dedicadas em nível de hardware para operar sem modificar o binário original.

1) Expansão do ISA: O hardware incorporou suporte nativo a três instruções requeridas pelo algoritmo:

- Instrução `sll`: o barramento do campo `instr[10:6]` foi conectado diretamente à porta `shamt` da ULA;
- Instrução `slti`: o opcode `001010` foi mapeado na controladora principal para forçar a operação `slt` da ULA sobre o operando imediato estendido;
- Instrução `bne`: o opcode `000101` foi integrado à lógica combinacional de desvio.

2) Bypass de Endereçamento do MARS: Trade-off de Engenharia: O montador MARS [8] aloca o segmento de código no endereço virtual canônico `0x00400000`, pressupondo uma infraestrutura completa de sistema operacional com suporte a memória virtual. Em uma implementação acadêmica *bare-metal* sintetizada diretamente em FPGA, a concepção de uma unidade de gerenciamento de memória (MMU) ou tabela de translação acrescentaria complexidade desnecessária e latência combinacional proibitiva no caminho crítico de busca.

Adotou-se, portanto, um *trade-off* temporário de engenharia em nível de hardware, descartando os 22 bits mais significativos via máscara combinacional na memória ROM:

```
assign instr = rom[addr[9:2]]; // Indexa palavras locais
```

Essa decisão técnica não constitui uma solução definitiva para sistemas de propósito geral, pois introduz sobreposição de endereços (*aliasing*) caso o espaço endereçável ultrapasse a capacidade física local de 256 palavras. Todavia, representa a escolha ótima para a Segunda Avaliação, garantindo latência zero, consumo nulo de recursos lógicos adicionais e execução fiel do binário original sem qualquer modificação em software.

3) Emulação de Ambiente de Execução (C-Runtime/SO Loader) no Hardware Bare-Metal: Em sistemas computacionais convencionais sob controle de um sistema operacional (SO), a carga e a execução de um binário envolvem rotinas de baixo nível providas pelo carregador do SO (*program loader*) e pelo código de inicialização do C-Runtime (`crt0.o`). Essas entidades configuram o ambiente de execução antes da invocação do programa: alocam espaço na memória de dados, preenchem vetores globais ou estáticos, inicializam o ponteiro de pilha (\$sp) no topo da memória do processo e carregam os parâmetros formais nos registradores de argumento (\$a0, \$a1) conforme a convenção de chamadas (MIPS ABI) [1].

No cenário *bare-metal* sobre FPGA, a ausência de um sistema operacional e de um montador com biblioteca de *runtime* impõe que o próprio hardware assuma o papel do SO Loader e do C-Runtime. Para que o binário `sort.bin` operasse de forma estritamente autônoma, essa emulação foi materializada em dois pontos complementares da arquitetura, conforme detalhado na Fig. 4:

1) **C-Runtime na Memória de Dados (`data_mem.v`)**: o bloco `initial` pré-aloca o vetor não ordenado $[88, 15, 42, 7, 99]$ nas primeiras cinco posições contíguas da RAM (`ram[0]` a `ram[4]`), emulando a carga do segmento de dados estáticos;

2) **SO Loader no Registrador (`reg_file.v`)**: durante o pulso de reset síncrono, a lógica interna pré-configura os registradores arquiteturais da ABI:

$$\text{sp} \leftarrow 256, \quad \text{a0} \leftarrow 0, \quad \text{a1} \leftarrow 5 \tag{5}$$

fixando o topo da pilha no limite da memória RAM (`$sp` = 256) e passando os parâmetros da chamada nos registradores de argumento `$a0` (endereço 0) e `$a1` (tamanho 5).

Essa estratégia assegura que, ao ser liberado do reset com o contador de programa em zero, o processador encontre o estado arquitetural perfeitamente consistente com as pré-condições do software, viabilizando a ordenação imediata e sem intervenções externas.

```
// 1. C-Runtime: carga do array na RAM (data_mem.v)
initial begin
    ram[0] = 32'd88;
    ram[1] = 32'd15;
    ram[2] = 32'd42;
    ram[3] = 32'd7;
    ram[4] = 32'd99;
end

// 2. SO Loader: bootloader no reset (reg_file.v)
registers[29] <= 32'd256; // $sp no topo da RAM
registers[4]  <= 32'd0;   // $a0 = base do vetor (0)
registers[5]  <= 32'd5;   // $a1 = tamanho (5)
```

Figura 4. Emulação de C-Runtime e SO Loader no hardware *bare-metal* (`data_mem.v` e `reg_file.v`).

IV. Validação Funcional e Resultados Experimentais

A metodologia de verificação adotada baseia-se no princípio de que nenhum componente deve ser integrado ao topo hierárquico sem prévia homologação funcional unitária. Foram desenvolvidas 7 bancadas de teste automatizadas com assertivas em Icarus Verilog [7], além da compilação física no Intel Quartus Prime [6].

A. Esteira de Automação e Metodologia de Testes

O fluxo unitário emprega o script `run_simulation.sh`, que aciona o simulador Icarus Verilog com suporte ao padrão IEEE 1800-2012 (`-g2012`) e ativação de avisos (`-Wall`). Essa rotina assegura a detecção precoce de sinais não conectados.

A execução da suíte completa ocorre pelo script de automação `run_all_tests.sh` via comando `make test`, compilando os módulos no motor `vvp`. A Tabela I consolida os resultados dos 7 módulos com 48 testes e 100% de aprovação.

B. Alvos de Verificação dos Módulos Unitários

Cada bancada de teste foi projetada com alvos de homologação específicos:

- **ULA (`tb_alu.v`)**: validou operações lógicas AND, OR e NOR, adições e subtrações com inteiros positivos e negativos em complemento a dois, ativação da linha de zero em resultado nulo, comparações de magnitude com sinal via `$signed` e deslocamento lógico `sll` com controle de deslocamento via `shamt`;
- **Datapath Auxiliar (`tb_datapath_aux.v`)**: comprovou o sincronismo do registrador de PC com reset síncrono, os somadores dedicados de PC + 4 e de branch com cálculo de deslocamento relativo, multiplexadores de seleção de barramentos e extensão de sinal com preservação do bit de magnitude;
- **Banco de Registradores (`tb_reg_file.v`)**: verificou a concorrência de leitura assíncrona em duas portas simultâneas, escrita síncrona habilitada por `reg_write` e blindagem inviolável do registrador $0, rejeitando tentativas de sobrescrita;
- **Unidade de Controle (`tb_control_unit.v`)**: testou a decodificação exata de instruções Tipo-R, Tipo-I e Tipo-J, os sinais de controle de memória e desvio, além da trava combinacional que inibe o sinal `reg_write` na instrução `jr`;
- **Memória de Instruções (`tb_instr_mem.v`)**: validou a leitura sequencial das instruções compiladas do arquivo `sort.bin` e a máscara de endereçamento `addr[9:2]` para contornar o offset virtual do simulador MARS;
- **Memória de Dados (`tb_data_mem.v`)**: homologou escritas síncronas na borda de subida do clock com `mem_write`, leituras assíncronas com `mem_read` e isolamento de palavras adjacentes.

Tabela I
Consolidado da Suíte de Testes Automatizada (`make test`)

Módulo	Arquivo	Testes	Status
ULA	`tb_alu.v`	11	Aprovado
Datapath Auxiliar	`tb_datapath_aux.v`	8	Aprovado
Banco de Registradores	`tb_reg_file.v`	6	Aprovado
Unidade de Controle	`tb_control_unit.v`	13	Aprovado
Memória de Instruções	`tb_instr_mem.v`	5	Aprovado
Memória de Dados	`tb_data_mem.v`	4	Aprovado
Top-Level com Sort	`tb_mips_monociclo_top.v`	1	Aprovado
Total da Suíte	**7 Arquivos**	**48**	**100% (0 erros)**

C. Resultados da Integração Top-Level e Ordenação

O teste definitivo da microarquitetura consistiu na execução do programa de ordenação canônico de Patterson e Hennessy [1] sobre o processador completo.

No início da simulação injetou-se na memória RAM um vetor de 5 elementos desordenados nas posições `ram[0]` a `ram[4]`. O processador operou de forma autônoma ao longo de 5000 ns de simulação, percorrendo laços de repetição, efetuando comparações e executando chamadas à sub-rotina de troca com `jal` e `jr`. A Tabela II exibe o estado físico das células de memória antes e depois da execução.

A transição do vetor de $[88, 15, 42, 7, 99]$ para a sequência ordenada $[7, 15, 42, 88, 99]$ comprova a integridade funcional do caminho de dados e da unidade de controle. A Fig. 6 exibe o log de execução do terminal.

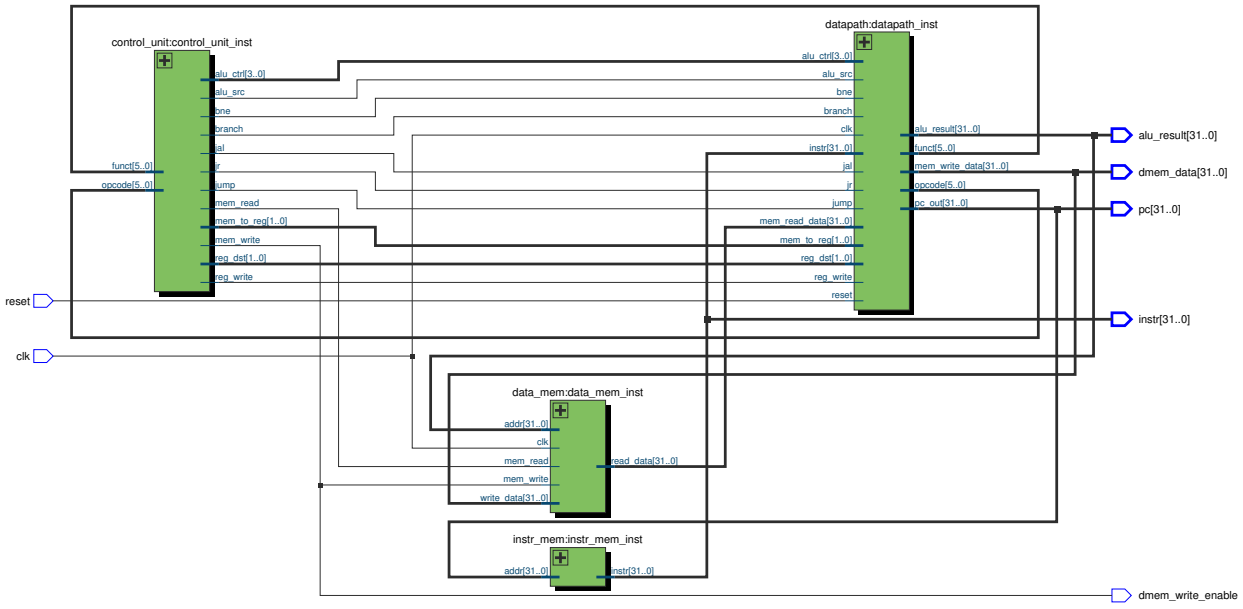


Figura 5. Netlist esquemático estrutural do processador MIPS monociclo no nível de topo (`mips_monociclo_top`) obtido via RTL Viewer no Intel Quartus Prime, evidenciando o modelo Harvard com barramentos desacoplados entre memória de instruções (`instr_mem`), memória de dados (`data_mem`), unidade de controle (`control_unit`) e caminho de dados (`datapath`).

Tabela II
ESTADO DA MEMÓRIA DE DADOS (RAM) ANTES E APÓS O SORT

Índice	Endereço	Inicial	Final	Posição
`ram[0]`	`0x00`	88	**7**	Mínimo
`ram[1]`	`0x04`	15	**15**	2° menor
`ram[2]`	`0x08`	42	**42**	Mediano
`ram[3]`	`0x0C`	7	**88**	4° elemento
`ram[4]`	`0x10`	99	**99**	Máximo

```
[ANTES]  RAM: [88, 15, 42, 7, 99]
[DEPOIS] RAM: [7, 15, 42, 88, 99]
>>> TESTE DO SORT CONCLUIDO COM SUCESSO! <<<
```

Figura 6. Log da simulação do processador executando o algoritmo Sort.

D. Validação Física e Síntese no Intel Quartus Prime

Para comprovar que a descrição em Verilog HDL representa um circuito realizável em silício e não apenas uma simulação abstrata, o projeto foi submetido ao fluxo completo de compilação física no Intel Quartus Prime [6] voltado ao FPGA Intel MAX 10 modelo `10M50DAF484C7G` da placa DE10-Lite [5].

O fluxo de compilação física via `make quartus-build` articula as seguintes etapas:

1) `quartus_map`: executou a análise sintática, elaboração hierárquica e síntese lógica dos 15 módulos HDL, mapeando o circuito em 17.089 elementos lógicos (LEs);
2) `quartus_fit`: realizou o posicionamento e roteamento físico nas matrizes de interconexão do chip com ocupação média de 15% dos recursos de roteamento. A ferramenta promoveu automaticamente os sinais `clk` e `reset` para as redes de clock globais dedicadas `CLKCTRL_G3` e `CLKCTRL_G4`, eliminando distorções de fase (*skew*) e prevenindo violações temporais;
3) `quartus_asm`: gerou o arquivo de programação binária `mips_monociclo.sof` com contagem nula de erros.

A Tabela III consolida as métricas da implementação física, cujo netlist sintetizado é detalhado na Fig. 5. A inspeção estrutural no RTL Viewer confirmou a ausência total de latches espúrios na ULA, nos multiplexadores e na unidade de controle, comprovando conformidade com as regras de projeto síncrono para lógica reconfigurável.

Tabela III
MÉTRICAS DA SÍNTESE FÍSICA NO INTEL QUARTUS PRIME

Métrica de Hardware	Resultado Obtido
Dispositivo Alvo	Intel MAX 10 (`10M50DAF484C7G`)
Elementos Lógicos (LEs)	17.089 células lógicas
Pinos de Entrada/Saída	131 pinos (2 entradas, 129 saídas)
Redes Globais de Relógio	2 (`CLKCTRL_G3` e `CLKCTRL_G4`)
Taxa de Roteamento (Interconnect)	15% média (75% pico regional)
Latches Transparentes Espúrios	0 inferidos (circuito 100% síncrono)
Bitstream de Programação	`mips_monociclo.sof` (0 erros)

A prevenção de latches espúrios constitui um dos critérios mais rigorosos no projeto de circuitos digitais sintetizáveis. Em descrições comportamentais com blocos `always @(*)`, a omissão involuntária de ramos condicionais em cláusulas `case` ou `if/else` induz a ferramenta a inferir elementos de memória transparentes assíncronos, degradando a confiabilidade do circuito. A análise do log de síntese confirmou cobertura exaustiva de ramos nas matrizes de decodificação de controle e multiplexação, assegurando lógica puramente

combinacional.

No estágio de roteamento (`quartus_fit`), o compilador promoveu os sinais `clk` e `reset` às linhas de distribuição de baixa impedância `CLKCTRL_G3` e `CLKCTRL_G4`. Essa atribuição minimiza distorções de fase (*clock skew*) sobre os 32 registradores e o contador de programa. A geração do bitstream binário `mips_monociclo.sof` comprova a prontidão do hardware para execução física imediata na placa DE10-Lite via comando `make quartus-pgm`.

V. Considerações Finais

A. Síntese do Atendimento aos Requisitos do Projeto

A Tabela IV sintetiza o atendimento formal aos requisitos técnicos e práticos estabelecidos para esta etapa do projeto.

Tabela IV
Síntese do Atendimento aos Requisitos do Projeto

Item	Requisito Técnico	Status
Item 1	Configuração do ambiente (Quartus e ferramentas).	100%
Item 2	Caminho de dados com saltos (`j`, `jal`, `jr`).	100%
Item 3	Validação de todos os módulos via testbenches.	100%
Item 4	Parametrização integral (`parameter`).	100%
Item 5	Apresentação em vídeo e relatório no padrão IEEE.	100%

B. Conclusão e Trabalhos Futuros

O desenvolvimento da Segunda Avaliação consolidou a implementação de um processador MIPS Monociclo de 32 bits em Verilog HDL. O hardware incorporou suporte às instruções `sll`, `slti` e `bne` para viabilizar a execução do algoritmo `sort.bin`. Além disso, implementou-se o mascaramento de barramento na memória ROM e a inicialização de registradores no reset para contornar o modelo de memória virtual do simulador MARS.

A validação automatizada comprovou a ordenação do vetor na memória de dados com zero falhas através de 7 bancadas de teste. A síntese lógica no Intel Quartus Prime para o FPGA Intel MAX 10 concluiu sem latches espúrios e com total coerência estrutural.

Na Terceira Avaliação Parcial o caminho de dados será segmentado nos cinco estágios clássicos de pipeline: busca, decodificação, execução, memória e escrita. Essa etapa introduzirá registradores de desacoplamento temporal, unidade de detecção de conflitos e unidade de adiantamento de operandos.

Referências

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Waltham, MA: Morgan Kaufmann, 2014.

[2] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA: Morgan Kaufmann, 2013.

[3] T. Brito, "Instruções: Linguagem do computador," Notas de Aula, Universidade Federal do Amazonas (UFAM), Manaus, 2026.

[4] *IEEE Standard for Verilog Hardware Description Language*, Institute of Electrical and Electronics Engineers Std. IEEE Std 1364-2005, 2006.

[5] Terasic Technologies, *DE10-Lite User Manual: Intel MAX 10 FPGA*, 1st ed., Terasic Inc., 2017.

[6] Intel Corporation, *Intel Quartus Prime Standard Edition User Guide: Design Compilation*, 20th ed., Intel Corporation, 2020, documento UG-20131.

[7] S. Williams, "Icarus Verilog Simulation and Synthesis Tool," http://iverilog.icarus.com/, 2020.

[8] K. Vollmar and P. Sanderson, "MARS: An Education-Oriented MIPS Assembly Language Simulator," in *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education*. ACM, 2006, pp. 239–243.